Algorithms used in the DTBench pipeline

LLM-Based Generation and Validation of Decision Table Test Cases

Overview

Step	Algorithm	Source file
Executable decision table	Algorithm 1	`dtbench/table.py`, `dtbench/expr.py`
Probe synthesis and structural check	Algorithm 2	`dtbench/probes.py`, `dtbench/validator.py`
Repair loop and test derivation	Algorithm 3	`dtbench/runner.py`, `dtbench/derive.py`
Q-learning decision table (P3)	Algorithm 4, Eq. (1)	`dtbench/rl/env.py`, `qlearning.py`
Mutation analysis	Eq. (5)	`dtbench/mutation.py`
Test-suite metrics	Eqs. (2)–(4)	`dtbench/test_eval.py`
Table metrics against ground truth	Eqs. (6)–(7)	`dtbench/validator.py`

Algorithms

Algorithm 1 Unique-hit evaluation of an executable decision table

Require: table $T = (C, R)$ with conditions C (boolean predicates over the inputs) and rules R; input x
Ensure: an output value, GAP, or CONFLICT

for each condition $c \in C$ **do**
  $v[c] \leftarrow c(x)$ ▷ safe AST evaluation; error ⇒ undefined
end for
$F \leftarrow \{\, r \in R \mid \forall (c, b) \in r.when : v[c] = b \,\}$ ▷ omitted conditions are don't-care
if $F = \emptyset$ **then**
  return GAP
end if
$O \leftarrow \{\, r.out \mid r \in F \,\}$
if $|O| > 1$ **then**
  return CONFLICT(F)
end if
return the single element of O ▷ overlaps with equal outputs are redundant, not inconsistent

Algorithm 2 Ground-truth-free structural validation with predicate-guided probes

Require: table $T = (C, R)$; data dictionary D (input types, domains $[\ell_x, h_x]$, constraint κ)
Ensure: list of issues I (malformed elements, missed combinations, conflicts)

$I \leftarrow$ malformed conditions, unknown condition references, outputs outside D
for each input variable x of D **do** ▷ candidate values
  $V_x \leftarrow$ all values if x is boolean/enumerated, else $\{\ell_x, \ell_x{+}1, \lfloor(\ell_x{+}h_x)/2\rfloor, h_x{-}1, h_x\}$
  add $k-\delta$, k, $k+\delta$ for every comparison $x \circ k$ in C; members of every set test $x \in S$; multiples of m and their neighbours for every test on $x \bmod m$
end for
$P \leftarrow$ sample of at most $N_\times$ points from $\prod_x V_x$
for each comparison $e_1 \circ e_2$ in C over several variables **do** ▷ relation solving
  for N_{rel} random base points p **do**
    for each variable y in which $e_1 - e_2$ is linear **do**
      solve $e_1(p[y{:=}y^*]) = e_2(p[y{:=}y^*])$; add $p[y{:=}y^*]$ and $p[y{:=}y^* \pm \delta]$ to P
    end for
  end for
end for
$P \leftarrow \{\, p \in P \cup \text{Uniform}(N_{\text{rnd}}) \mid p \text{ in domains},\ \kappa(p) \,\}$
for each distinct condition vector v realised by some $p \in P$ **do**
  $F \leftarrow$ rules matching v
  if $F = \emptyset$ **then**
    add *missed combination* (v, p) to I
  else if rules in F give different outputs **then**
    add *conflict* (F, p) to I
  end if
end for
return I

Algorithm 3 Validation-guided repair (P2) and test derivation

Require: business rules B; data dictionary D; model M; maximum rounds K $(= 2)$
Ensure: decision table T and test suite S

$T \leftarrow M(\textsc{TablePrompt}(B, D))$ ▷ greedy decoding
for $k = 1$ to K **do**
  $I \leftarrow \textsc{StructuralCheck}(T, D)$ ▷ Algorithm 2
  if $I = \emptyset$ **then**
    break
  end if
  $T' \leftarrow M(\textsc{RepairPrompt}(B, D, T, I))$ ▷ at most 6 issues with witness inputs
  if T' is not valid JSON **then**
    break
  end if
  $T \leftarrow T'$
end for
$Q \leftarrow$ uniform random inputs $\cup$ probes of T's conditions ▷ same procedure for every table
$S \leftarrow \emptyset$
for each usable rule r of T **do**
  $q \leftarrow$ first $q \in Q$ whose condition vector matches $r.when$
  if q exists **then**
    $S \leftarrow S \cup \{(q,\ r.out)\}$
  end if
end for
return T, S

Algorithm 4 Q-learning of a decision table (P3)

Require: conditions C of the LLM's table; system under test f (gives the reward); episodes E; α, γ; exploration ε_0, $\varepsilon_{\min}$, decay d
Ensure: Q-table Q and learned decision table T_Q

$X \leftarrow$ probes of C (Algorithm 2) $\cup$ uniform random inputs
$S \leftarrow$ distinct condition vectors $v_C(x)$, $x \in X$; $A \leftarrow$ allowed outputs
$Q[s,a] \leftarrow 0$ for all $s \in S$, $a \in A$; $\varepsilon \leftarrow \varepsilon_0$
for $e = 1$ to E **do** ▷ one episode = every state once
  for each state $s \in S$ in random order **do**
    $x \leftarrow$ random input in X with $v_C(x) = s$
    $a \leftarrow$ random with prob. ε, else $\arg\max_{a'} Q[s,a']$
    $r \leftarrow +1$ if $a = f(x)$, else -1
    $Q[s,a] \leftarrow Q[s,a] + \alpha\,(r + \gamma \max_{a'} Q[s',a'] - Q[s,a])$
  end for
  $\varepsilon \leftarrow \max(\varepsilon_{\min},\ d\,\varepsilon)$
end for
$T_Q \leftarrow$ one rule per state s: entries of s, output $\arg\max_a Q[s,a]$
return Q, T_Q

Q-learning (P3)

The Gymnasium environment (`dtbench/rl/env.py`) shows one test input per step. The state s is the combination of the LLM's conditions that the input falls in, the action a is an output value, and the reward is $r = +1$ if a equals the output of the system under test and $r = -1$ otherwise. One episode visits every state once (one epoch). The agent updates its Q-table with

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[\, r + \gamma \max_{a'} Q(s',a') - Q(s,a) \,\big] \tag{1}$$

and explores ε-greedily. Settings (file `config/rl_config.json`): 150 episodes, $\alpha = 0.1$, $\gamma = 0$ (the next input does not depend on the answer), ε from 1.0 to 0.05 with decay 0.97 per episode, rewards ± 1, seed 42. With $\gamma = 0$, $Q(s,a)$ tends to $2p - 1$, where p is the share of inputs in state s for which the system returns a, so the greedy decision is the most frequent output.

Metrics

Let R be the feasible rules of the ground-truth table, P the evaluation probes, ref the reference implementation, S a test suite and $S^+ \subseteq S$ the valid tests whose expected output equals ref. For an input x, $\rho(x)$ is the unique ground-truth rule that x satisfies.

$$\mathrm{RC}(S) = \frac{|\{\rho(x) \mid (x,y) \in S^+\}|}{|R|} \tag{2}$$

$$\mathrm{Missed}(S) = R \setminus \{\rho(x) \mid (x,y) \in S^+\} \tag{3}$$

$$\mathrm{OracleAcc}(S) = \frac{|S^+|}{|S_{\mathrm{valid}}|} \tag{4}$$

$$\mathrm{MS}(S) = \frac{|\{m \in \mathcal{M} \setminus \mathcal{E} \mid \exists (x,y) \in S^+ : m(x) \neq y\}|}{|\mathcal{M} \setminus \mathcal{E}|} \tag{5}$$

$$\mathrm{Acc}(T) = \frac{|\{p \in P \mid \mathrm{eval}(T,p) = \mathrm{ref}(p)\}|}{|P|} \tag{6}$$

$$\mathrm{GapRate}(T) = \frac{|\{p \in P \mid \mathrm{eval}(T,p) = \textsc{Gap}\}|}{|P|}, \quad \mathrm{ConflictRate}(T) = \frac{|\{p \in P \mid \mathrm{eval}(T,p) = \textsc{Conflict}\}|}{|P|} \tag{7}$$

$\mathcal{M}$ are the first-order mutants of the reference implementation (operators ROR, LCR, AOR, CRP, RVR, NEG, NDL); $\mathcal{E}$ are mutants that behave identically to the original on every evaluation probe (treated as equivalent).

Complexity

For a table with $|C|$ conditions and $|R|$ rules evaluated on $|P|$ probes, Algorithm 1 costs $O(|C| + |R|\,|C|)$ per input, so validation costs $O(|P|\,|R|\,|C|)$ after probes are grouped by condition vector (at most $2^{|C|}$ distinct vectors, in practice far fewer). Probe synthesis is bounded by the parameters $N_\times = 6000$, $N_{\mathrm{rel}} = 50$, $N_{\mathrm{rnd}} = 1200$ (validation) and 20000/250/4000 (evaluation probes).